Microservices Architecture

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

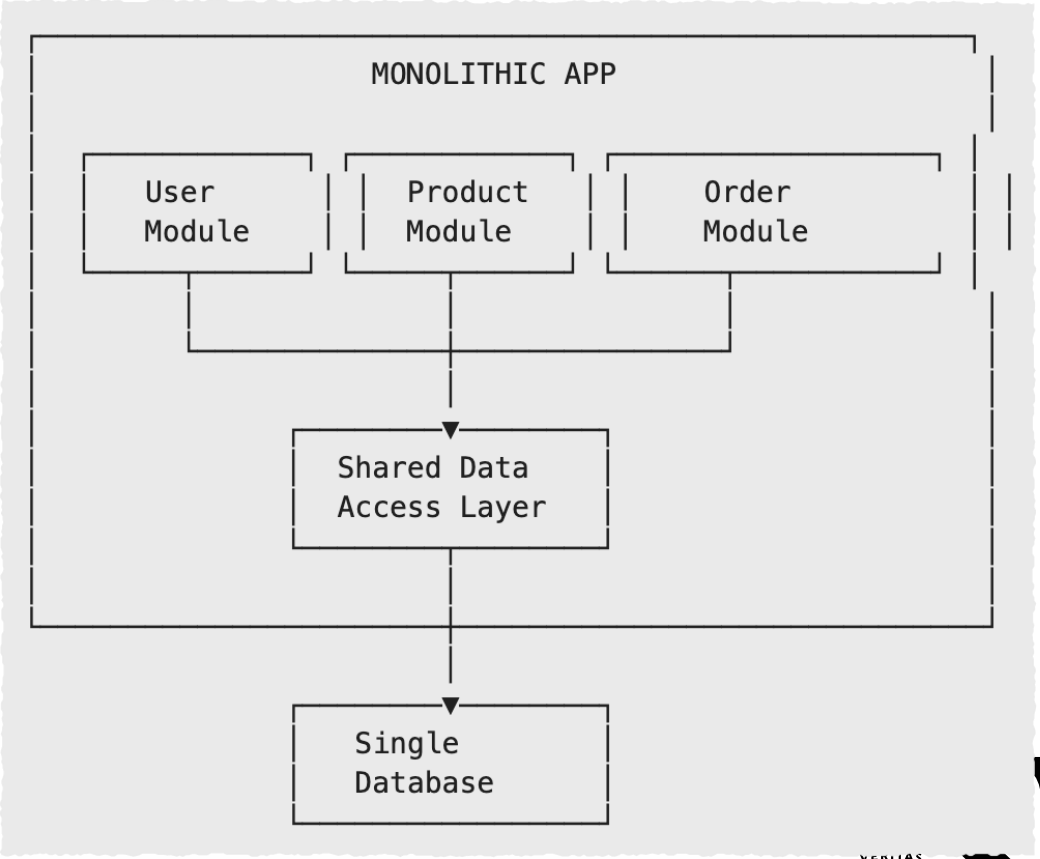
Learning Objectives

- Explain the difference between monolithic and microservices architectures and articulate when each is appropriate.
- Identify the core principles that define a well-designed microservice: single responsibility, loose coupling, and high cohesion.
- Describe common integration patterns used between microservices, including REST, message queues, and event-driven communication.
- Map microservices concepts to specific AWS and GCP services used to build, deploy, and manage them.
- Recognize the operational challenges of microservices — data consistency, observability, network complexity — and explain the cloud-native tooling that addresses them.

The Monolith: Where We Start

What Is a Monolithic Application?

- A monolithic application packages all of its functionality — the user interface, business logic, data access layer, and background jobs — into a single deployable artifact. Think of a Java WAR file, a Ruby on Rails app, or a single Node.js Express application that handles everything.
- In a monolith, all modules share the same process memory, the same database, and the same deployment lifecycle. When you deploy, you deploy everything. When you scale, you scale everything.



Advantages of a Monolith

Monoliths are not inherently bad. For small teams and early-stage projects, they offer real advantages:

- **Simple to develop.** One codebase, one IDE project, one build. A new developer can clone a single repository and start working.
- **Simple to test.** End-to-end tests run against one process. No network calls to mock.
- **Simple to deploy.** Copy one artifact to a server. One CI/CD pipeline, one rollback procedure.
- **Simple to debug.** A stack trace shows you the full call chain. You can step through code from the HTTP handler all the way to the database query.
- **No network overhead between modules.** Function calls within a single process are measured in nanoseconds, not milliseconds.

When Monoliths Break Down

As the application grows — more features, more developers, more traffic — the monolith starts to strain:

- **Codebase bloat.** The repository grows to hundreds of thousands of lines. Finding where a specific piece of logic lives becomes a scavenger hunt. A bug in the payment module might be caused by a change someone made in the notification module weeks ago.
- **Deployment risk.** Every deployment ships the entire application. A one-line CSS change requires redeploying the billing engine. If something goes wrong, you roll back everything.
- **Scaling is all-or-nothing.** If the image processing module needs 10x more CPU but the user profile module is idle, you still scale the whole application. You cannot independently scale the part that needs it.
- **Technology lock-in.** The entire application must use the same language, framework, and runtime version. Upgrading from Java 8 to Java 17 means upgrading the entire application at once.
- **Team coupling.** Multiple teams working on the same codebase create merge conflicts, coordination overhead, and deployment bottlenecks. A team that is ready to ship waits for another team that is not.

Monolith Scaling Problem

```
Monolith Scaling Problem
═══════════════════════

Only the Order module is under heavy load,
but you must scale the ENTIRE application:

┌──────────┐    ┌──────────┐    ┌──────────┐
│ Instance │    │ Instance │    │ Instance │
│    1     │    │    2     │    │    3     │
│          │    │          │    │          │
│ [Users]  │    │ [Users]  │    │ [Users]  │
│ [Product]│    │ [Product]│    │ [Product]│  ← Wasted capacity
│ [Orders] │    │ [Orders] │    │ [Orders] │  ← Only this needs scaling
│ [Ship]   │    │ [Ship]   │    │ [Ship]   │  ← Wasted capacity
└──────────┘    └──────────┘    └──────────┘
      ▲               ▲               ▲
      └───────────────┼───────────────┘
                      │
                      ▼
              ┌──────────────┐
              │ Load Balancer │
              └──────────────┘
```

Microservices

Core Concepts

Definition

Microservices architecture structures an application as a collection of loosely coupled, independently deployable services. Each service is organized around a specific business capability, owns its own data, and communicates with other services over the network.

```
Microservices Architecture
═══════════════════════════

┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
│ User     │   │ Product  │   │ Order    │   │ Shipping │
│ Service  │   │ Service  │   │ Service  │   │ Service  │
│          │   │          │   │          │   │          │
│ [API]    │   │ [API]    │   │ [API]    │   │ [API]    │
└──────────┘   └──────────┘   └──────────┘   └──────────┘
     │              │              │              │
     ▼              ▼              ▼              ▼
┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
│ User DB  │   │Product DB│   │ Order DB │   │ Ship DB  │
│(Postgres)│   │ (Mongo)  │   │ (MySQL)  │   │(Dynamo)  │
└──────────┘   └──────────┘   └──────────┘   └──────────┘

Each service:
 • Owns its data store
 • Deploys independently
 • Can use different technology
 • Communicates via network APIs
```

Key Characteristics

- **Small and focused on doing one thing well**
  - A microservice is built around a single business capability. The "User Service" handles user registration, authentication, and profile management. It does not handle order processing. The boundary of the service aligns with the boundary of the business domain.
  - As a codebase grows, it becomes harder to know where to make changes. Code for related functionality drifts apart across modules and packages. Microservices address this by drawing hard boundaries: all the code for user management lives in one service, with one repository, one build pipeline, and one team responsible for it.
  - How small is "small"? There is no universal rule. Some teams use the "two-pizza team" heuristic — a service should be small enough for a team of 5–8 people to own it fully. Others use the "can you rewrite it in two weeks" test. The important thing is that the service has a clear, focused purpose.
- **Autonomous**
  - Each microservice is a separate entity — a separate process, often a separate container, deployed independently. All communication between services happens over the network, enforcing a hard boundary between them.
  - This autonomy means a team can change, test, and deploy their service without coordinating with every other team. A new version of the User Service can go to production without touching the Order Service. This is possible because services interact through well-defined APIs, and as long as the API contract does not break, internal changes are invisible to consumers.

The Two Pillars of Good Service Design

- **Loose Coupling.** When services are loosely coupled, a change to one service does not require a change to another. If deploying a new version of the Shipping Service forces you to also update the Order Service, those services are tightly coupled. Loose coupling means services know as little as possible about each other's internal implementation.
- **High Cohesion.** Related behavior should live together. If you need to change how shipping costs are calculated, that change should be confined to the Shipping Service. You should not need to touch three different services to implement a single business change. High cohesion means the service encapsulates all the logic and data for its domain.

```
Loose Coupling vs. Tight Coupling
═══════════════════════════════════

LOOSE COUPLING (good):              TIGHT COUPLING (bad):

┌──────────┐      HTTP/API      ┌──────────┐
│ Service  │────────────────────│ Service  │
│    A     │   "I only          │    B     │
└──────────┘    know your       └──────────┘
                 API"

Change B's internals → A is unaffected

┌──────────┐                    ┌──────────┐
│ Service  │────────────────────│ Service  │
│    A     │     Shared DB      │    B     │
└──────────┘                    └──────────┘
     │                               │
     └────┌──────────┐───────────────┘
          │ Shared   │
          │ Database │
          └──────────┘

Change B's schema → A breaks
```

Benefits of Microservices in the Cloud

Benefits

- Cloud platforms were designed for the kind of workloads microservices produce: many small, independently scalable, loosely coupled services.
- The benefits of microservices become significantly more powerful when deployed on cloud infrastructure.

Independent Scaling

- In a monolith, you scale the entire application even if only one module is under load. With microservices on the cloud, you scale individual services based on their specific demand.
- If the Order Service sees a 10x traffic spike during a flash sale, you scale just that service. The User Service stays at its current capacity. AWS Auto Scaling groups, ECS service auto-scaling, and GCP's Cloud Run automatic scaling all make this straightforward. You pay only for what each service actually consumes.

```
Independent Scaling with Microservices
═══════════════════════════════════════

Normal traffic:                 Flash sale:

[User]  x2 instances            [User]  x2 instances  (unchanged)
[Order] x2 instances    ──►     [Order] x8 instances  (auto-scaled)
[Ship]  x2 instances            [Ship]  x3 instances  (slight bump)

Cloud auto-scaling adjusts each service independently
based on CPU, memory, request count, or custom metrics.
```

Technology Heterogeneity

- Each service can use the technology stack best suited to its problem domain. The Product Catalog service might use a document database like MongoDB or DynamoDB because products have varied attributes. The Order service might use a relational database like PostgreSQL or RDS for transactional integrity. The Search service might use Elasticsearch or OpenSearch. The Recommendation engine might be written in Python for its machine learning libraries while the real-time Bidding service is in Go for performance.
- In a monolith, all modules are locked to one language, one framework, and one database. Microservices remove this constraint entirely.

Resilience and Fault Isolation

- A well-designed microservices architecture contains failures. If the Recommendation Service goes down, customers can still browse products, add them to carts, and check out. The system degrades gracefully instead of failing entirely.
- Cloud platforms reinforce this with health checks, automatic restarts, circuit breakers, and multi-AZ deployments. AWS ECS or GCP Cloud Run will automatically replace an unhealthy container. A load balancer will stop sending traffic to a failing instance.
- In a monolith, an unhandled exception or memory leak in one module can crash the entire application.

Independent Deployments

- Each service has its own CI/CD pipeline. A team can build, test, and deploy their service multiple times a day without coordinating with other teams. This dramatically reduces deployment risk and increases velocity.
- If something goes wrong, you roll back one service — not the entire platform. Canary deployments and blue-green deployments work at the individual service level, making safe deployments more practical.

Organizational Alignment

Microservices align well with Conway's Law — the observation that system architecture tends to mirror organizational communication structures. If you have a team responsible for payments, they own the Payment Service end-to-end: the code, the database, the deployment pipeline, the on-call rotation. This clear ownership reduces ambiguity and improves accountability.

Composability

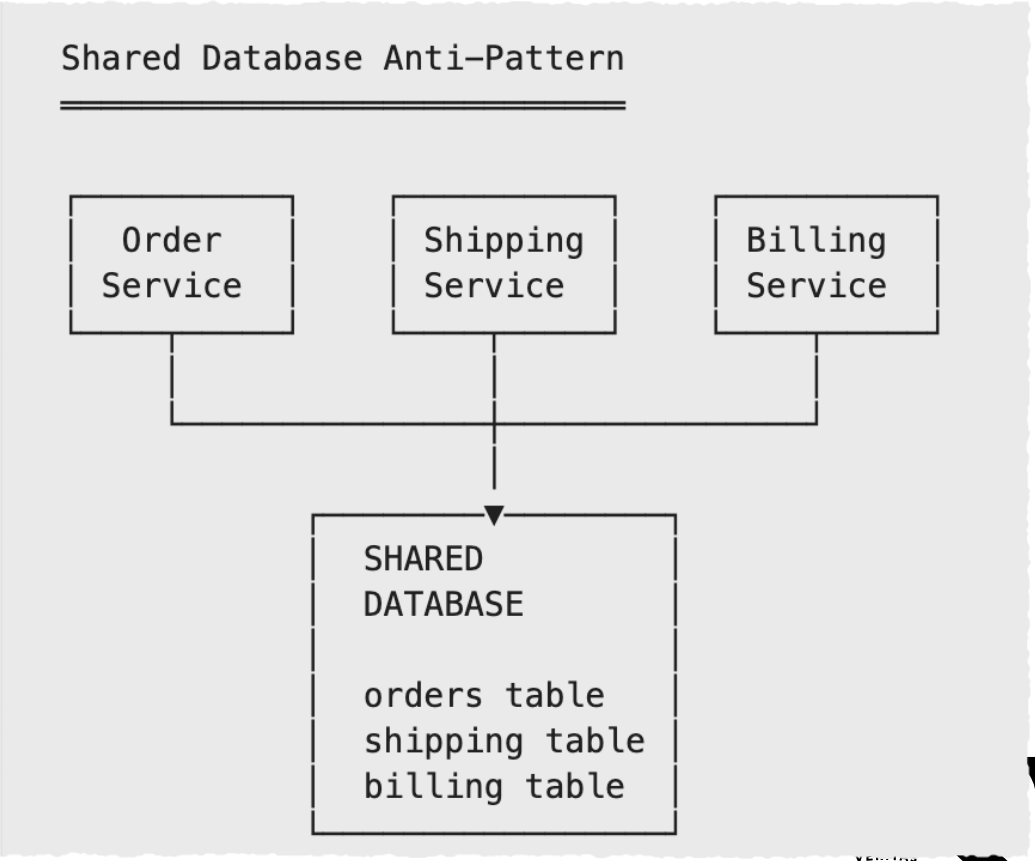
Microservices expose APIs that can be consumed by different clients for different purposes. The same Product Service API might be consumed by the web storefront, the mobile app, an internal admin dashboard, and a partner integration. The service does not care who is calling it — it provides a capability, and consumers compose those capabilities into experiences.

Integration Patterns

Services need to talk to each other. How they communicate is one of the most consequential architectural decisions in a microservices system. There are several patterns, each with different trade-offs.

Shared Database (Anti-Pattern)

- The most tempting and most dangerous integration pattern.
- Multiple services read from and write to the same database.
- Problems:
  - Schema change in one service breaks others
  - No enforcement of service boundaries
  - Tight coupling through shared data
  - Cannot independently scale data stores
  - All services locked to one database technology
- Why is this bad? Because it destroys the very property that makes microservices valuable — independence. If the Order Service and the Shipping Service both query the orders table directly, a schema change in that table requires coordinating changes across multiple services simultaneously. You are back to monolith-level coupling, hidden behind a network boundary.
- The rule: each service owns its data. No other service reads from or writes to another service's data store directly. If the Shipping Service needs order information, it calls the Order Service API.



Synchronous Communication: REST and RPC

- **REST (Representational State Transfer)** is the most common synchronous pattern. Services expose HTTP endpoints, and other services call them. This is the pattern your course assignments use — a web application exposing REST APIs.
- **RPC (Remote Procedure Call)** makes calling a remote service look like calling a local function. Frameworks like gRPC (used extensively at Google) use Protocol Buffers for efficient binary serialization and HTTP/2 for transport. RPC is faster than REST for service-to-service communication but adds tooling complexity.
- **Trade-offs of synchronous communication:**
  - Simple to understand and debug. Request goes out, response comes back.
  - Creates temporal coupling — the caller must wait for the response, and if the downstream service is slow or down, the caller is affected.
  - Can create cascading failures. If Service A calls Service B, which calls Service C, and Service C is slow, all three services are impacted.

```
Synchronous REST Communication
═══════════════════════════════

Order Service needs user details:

┌──────────────┐   GET /users/123   ┌──────────────┐
│   Order      │ ──────────────────→ │   User       │
│  Service     │                     │  Service     │
│              │ ←────────────────── │              │
└──────────────┘   200 OK + JSON    └──────────────┘

Order Service WAITS until User Service responds.
If User Service is slow, Order Service is slow.
If User Service is down, Order Service fails.
```

Asynchronous Communication: Message Queues and Events

- In asynchronous communication, the sender publishes a message and does not wait for a response.
- A message broker sits between services.
- Trade-offs of asynchronous communication:
  - Temporal decoupling — the sender does not wait, so a slow or down consumer does not impact the sender.
  - Natural fit for event-driven patterns and fan-out to multiple consumers.
  - More complex to reason about. You cannot just follow a request from start to finish. Events might be processed out of order. You need to handle duplicate deliveries (idempotency).
  - Harder to debug. "Why didn't the shipping label get created?" might require tracing a message through multiple queues.

```
Asynchronous Communication via Message Queue
═══════════════════════════════════════════

┌──────────┐   Publish   ┌──────────────┐   Consume   ┌──────────┐
│  Order   │ ──────────► │ Message Queue │ ──────────► │ Shipping │
│ Service  │ "OrderCreated" │ (SQS / Pub/Sub) │         │ Service  │
└──────────┘             └──────────────┘             └──────────┘
     │
     │  Order Service does NOT wait.
     │  It publishes the event and moves on.
     ▼
Responds to client immediately.
```

```
Event-Driven with Multiple Consumers
═══════════════════════════════════

┌──────────┐                 ┌──────────────┐
│  Order   │ ──────────────► │  Event Bus   │
│ Service  │ "OrderCreated" │ (SNS / EventBridge │
└──────────┘                 │  / Pub/Sub)  │
                             └──────┬───────┘
                                    │
                  ┌─────────────────┼─────────────────┐
                  ▼                 ▼                 ▼
            ┌──────────┐      ┌──────────┐      ┌──────────┐
            │ Shipping │      │ Billing  │      │ Notif.   │
            │ Service  │      │ Service  │      │ Service  │
            └──────────┘      └──────────┘      └──────────┘

One event, multiple consumers — each reacts independently.
```

Synchronous vs. Asynchronous: When to Use Each

- Use **synchronous** (REST/gRPC) when the caller needs an immediate response to proceed. For example, a web app needs to fetch a user's profile to render a page.
- Use **asynchronous** (queues/events) when the operation can happen in the background and the caller does not need to wait. For example, after an order is placed, sending a confirmation email and generating a shipping label can happen asynchronously.
- Many real systems use both: synchronous for reads (fetching data to display) and asynchronous for writes that trigger downstream workflows (order placed → notify warehouse → generate label → send email).

The API Gateway Pattern

- When you have dozens or hundreds of microservices, you do not want external clients (mobile apps, web browsers) calling each service directly.
- An API Gateway is a single entry point that routes requests to the appropriate backend service.
- Cloud-managed API gateways: **AWS API Gateway** and **GCP API Gateway** (or Cloud Endpoints). These handle routing, authentication, throttling, and monitoring without you writing custom code.

```
API Gateway Pattern
═══════════════════

┌─────────┐     ┌─────────┐
│ Mobile  │     │  Web    │
│  App    │     │Browser  │
└─────────┘     └─────────┘
     │               │
     └───────┬───────┘
             ▼
    ┌──────────────┐
    │ API Gateway  │    Handles:
    │              │     • Request routing
    │ (AWS API GW  │     • Authentication
    │  / GCP API   │     • Rate limiting
    │  Gateway)    │     • SSL termination
    └──────────────┘    • Request/response transformation
             │
     ┌───────┼───────┐
     ▼       ▼       ▼
┌─────┐   ┌─────┐   ┌─────┐
│User │   │Order│   │Product│
│Svc  │   │Svc  │   │Svc  │
└─────┘   └─────┘   └─────┘
```

Data Management in Microservices

Database per Service

- The foundational data management principle in microservices: each service owns its data store, and no other service accesses it directly. This means if you have five services, you might have five separate databases.
- This feels wasteful at first. Why not share one database? Because shared databases create hidden coupling. When the Order Service and the Inventory Service both depend on the same products table, changing that table requires coordinating both teams. Independent data stores give each team complete control over their schema, query patterns, indexing strategy, and even database technology.

The Data Consistency Challenge

- With separate databases, you lose traditional database transactions that span multiple tables. In a monolith, placing an order might look like one SQL transaction that atomically decrements inventory, creates an order record, and charges the customer. In microservices, these operations span different services with different databases. You cannot wrap them in a single ACID transaction.
- This brings us to **eventual consistency**: the system will become consistent over time, but at any given moment, different services might have temporarily inconsistent views of the world. When a customer places an order, the Order Service might record the order immediately, but the Inventory Service might take a few hundred milliseconds to decrement stock. During that window, the data is inconsistent — but it will resolve.

The Saga Pattern

- A saga is a sequence of local transactions across services, coordinated either by choreography (each service publishes events that trigger the next step) or orchestration (a central coordinator tells each service what to do).
- This is more complex than a single database transaction, but it is the reality of distributed systems. Cloud services like AWS Step Functions and GCP Workflows can help orchestrate sagas.

```
Saga Pattern: Order Placement (Choreography)
═══════════════════════════════════════════════

Step 1          Step 2              Step 3           Step 4

┌──────────┐    ┌──────────────┐    ┌──────────┐    ┌──────────┐
│  Order   │ ──►│  Inventory   │ ──►│ Payment  │ ──►│ Shipping │
│ Service  │    │   Service    │    │ Service  │    │ Service  │
│          │    │              │    │          │    │          │
│ Create   │    │ Reserve      │    │ Charge   │    │ Create   │
│ Order    │    │ Stock        │    │ Customer │    │ Shipment │
└──────────┘    └──────────────┘    └──────────┘    └──────────┘
      │               │                  │               │
      ▼               ▼                  ▼               ▼
 OrderCreated    StockReserved     PaymentCharged   ShipmentCreated
   (event)         (event)            (event)          (event)

If Step 3 fails (payment declined):
  → Payment Service publishes "PaymentFailed"
  → Inventory Service listens and releases reserved stock
  → Order Service listens and marks order as cancelled
  These are called "compensating transactions"
```

Drawbacks and Challenges of Microservices

Distributed System Complexity

Every call between services is a network call. Networks are unreliable. Requests can fail, time out, arrive out of order, or be delivered twice. You must handle all of these cases in code. In a monolith, a function call either succeeds or throws an exception. In microservices, a call might succeed on the remote end but the response is lost in transit — did it work or not?

Data Consistency

- You lose ACID transactions across services.
- Eventual consistency is harder to reason about.
- Sagas add complexity.
- Debugging a data inconsistency across five services with five databases is significantly harder than debugging one database.

Operational Overhead

Instead of operating one application, you operate dozens or hundreds. Each service needs its own CI/CD pipeline, monitoring, logging, alerting, and on-call rotation. Without strong automation and platform engineering, this overhead can overwhelm a team.

Network Latency

A function call in a monolith takes nanoseconds. An HTTP call between services takes milliseconds at minimum. If a single user request cascades through 10 services sequentially, those milliseconds add up. Careful API design, caching, and parallel calls are necessary to keep latency acceptable.

Testing Complexity

- Unit testing a single service is straightforward. Integration testing — verifying that services work together correctly — is much harder.
- You need to either run all dependent services locally (heavy) or maintain mock services (which can drift from reality).
- End-to-end testing of a microservices system requires significant infrastructure.

The "Distributed Monolith" Trap

- If your microservices must be deployed together, share a database, or require lockstep changes, you have a "distributed monolith" — all the complexity of microservices with none of the benefits.
- This is the worst possible outcome and typically results from poor service boundary design or the shared database anti-pattern.

When NOT to Use Microservices

- Microservices are not appropriate for every situation:
  - **Small teams (fewer than 5–10 developers).** The operational overhead is not justified.
  - **New or uncertain domains.** If you do not understand your business domain well enough to draw clear service boundaries, you will draw them wrong. Start with a monolith, learn the domain, then split.
  - **Prototypes and MVPs.** Speed of iteration matters more than architectural purity.
  - **Simple applications.** A basic CRUD application does not need microservices.
- Martin Fowler's advice: "Don't start with microservices. Start with a monolith, keep it modular, and split it up when you have a clear reason to."

Migration Strategies

The Strangler Fig Pattern

- Named after a tropical plant that grows around a tree and eventually replaces it, the Strangler Fig pattern incrementally migrates a monolith to microservices without a risky "big bang" rewrite.
- An API gateway or reverse proxy routes traffic: new requests go to the new microservice, remaining requests go to the monolith. Over time, more functionality moves into services until the monolith is empty and can be decommissioned.

```
Strangler Fig Pattern
═══════════════════

Phase 1: All traffic goes to monolith

┌──────────┐
│ Monolith │ ← handles everything
└──────────┘

Phase 2: New service handles one capability

┌──────────┐      ┌──────────┐
│ Monolith │      │ New      │
│ (minus   │      │ User     │
│ user mgmt)│     │ Service  │
└──────────┘      └──────────┘
     ▲                 ▲
     └─────────┬───────┘
               ▼
          ┌──────────┐
          │ API Gateway │  Routes /users to new service
          │ / Proxy     │  Routes everything else to monolith
          └──────────┘

Phase 3: More services extracted

┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│ Monolith │ │ User     │ │ Order    │ │ Product  │
│ (shrinking)│ │ Service │ │ Service  │ │ Service  │
└──────────┘ └──────────┘ └──────────┘ └──────────┘

Phase 4: Monolith is gone

┌──────────┐ ┌──────────┐ ┌──────────┐
│ User     │ │ Order    │ │ Product  │
│ Service  │ │ Service  │ │ Service  │
└──────────┘ └──────────┘ └──────────┘
```

Summary

Key Takeaways

- Microservices architecture decomposes an application into small, independently deployable services aligned with business capabilities. Each service owns its data, communicates over the network, and can be developed, tested, and scaled independently.
- The cloud amplifies microservices benefits — independent scaling, fault isolation, technology choice — while also providing the infrastructure needed to manage the added complexity: managed container platforms, service discovery, message queues, distributed tracing, and centralized logging.
- Microservices are not free. They trade monolith simplicity for distributed system complexity. Teams need mature CI/CD, observability, and platform engineering before the benefits outweigh the costs. The right starting point for most projects is a well-structured monolith, migrated incrementally as the need arises.

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS